30-Day Android Challenge

Senior Android interview prep — Junior → Mid → Senior → Architect

2026 Edition

DAY 15

Background Work — Part 1: WorkManager, Doze & App Standby



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 15

Why background work matters in 2026

Background work is where apps lose users' trust: a drained battery, a sync that never happened, a crash after a targetSdk bump. Interviewers use it to check whether you know **what the system will — and won't — let you do**:

- Work that must survive the process belongs to **WorkManager**, built on JobScheduler
- **Constraints, retries and unique work** instead of hand-rolled schedulers
- **Expedited work** for urgent, short tasks — paid for with a quota
- **Doze and App Standby buckets** decide *when* your work actually runs
- The rules change with almost every release — know **which version** introduced each limit

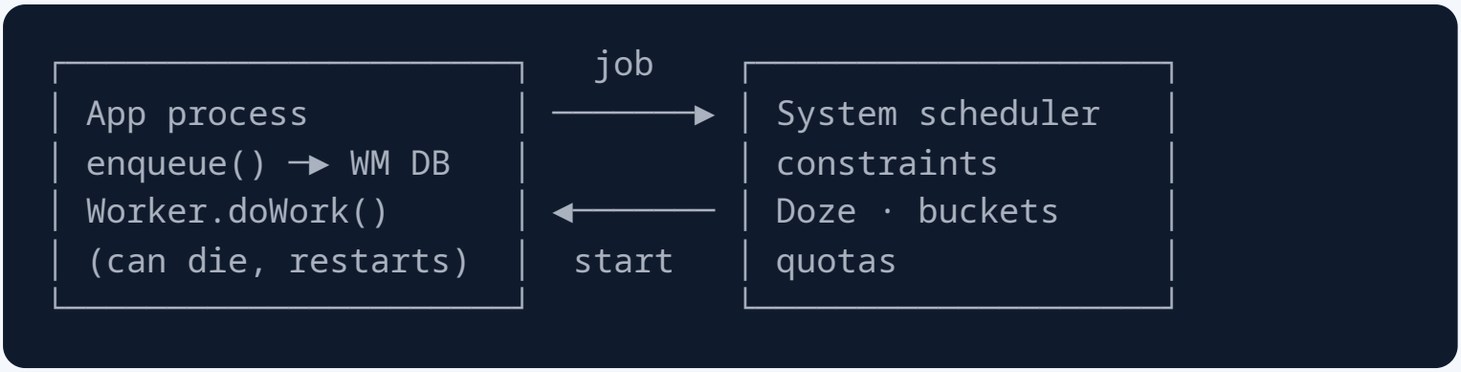
Level	Questions
Junior	What WorkManager is · Your first worker · Constraints & periodic work
Mid	Chaining · Unique work · Retries & backoff · Observing & testing
Senior	Expedited & long-running work · Doze & App Standby · Limits by version
Architect	A background-work layer · Battery-friendly sync · Observability



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 15

Mental model: your process is not the scheduler



- WorkManager **persists** the request, then hands it to the system scheduler (JobScheduler)
- Your process may die at any time; the **system** remembers the job and restarts your process to run it
- *When* it runs is decided by constraints, Doze, your standby bucket and quotas — **not by you**

THE ONE QUESTION BEHIND EVERY ANSWER TODAY

"What happens to this work if the process dies right now?" If the answer is "it's lost" and that's not acceptable, it doesn't belong in a coroutine.



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 15

Junior

What WorkManager is · Your first worker · Constraints & periodic work



Esmaeeil Moradi
@esmaeeil-moradi700

Q1. What WorkManager is — and what it isn't

WorkManager is Jetpack's API for **persistent work**: work that must still happen if the user leaves the app, the process is killed or the device reboots.

- Requests are stored in WorkManager's own database
- On API 23+ it schedules through **JobScheduler**; while your process is alive it can also start eligible work right away
- It re-schedules its work after a reboot — no boot receiver of your own needed

Need	Tool
Work tied to a screen (load data, save on click)	Coroutine in `viewModelScope`
Must finish even if the app is closed; can wait for conditions	**WorkManager**
User-visible and ongoing (playback, navigation, a call)	Foreground service — Day 16
Must fire at an exact clock time	Exact alarm — Day 16



Esmaeeil Moradi
@esmaeeil-moradi700

Q1. What interviewers check

INTERVIEW TRAP

"On old devices WorkManager uses Firebase JobDispatcher or GCMNetworkManager." — Outdated (last year's version said this too). Firebase JobDispatcher was deprecated years ago. Today: JobScheduler on API 23+, AlarmManager-based scheduling only below that.

INTERVIEW TRAP

"Guaranteed means it runs now — and always." — It means **eventually, when constraints allow**. If the user **force-stops** the app, the system cancels its jobs and alarms; nothing runs until the app is opened again (WorkManager then re-schedules). Some OEM builds are even more aggressive.

FOLLOW-UP

"Why not just launch a coroutine in an application scope?" — It dies with the process, ignores constraints and never retries. Fine for work you can afford to lose (warming a cache) — not for "must happen".



Esmaeeil Moradi
@esmaeeil-moradi700

Q2. Your first worker

```
class SyncWorker(
    context: Context,
    params: WorkerParameters,
    private val repo: SyncRepository, // WorkerFactory, Q11
) : CoroutineWorker(context, params) {

    override suspend fun doWork(): Result {
        val userId = inputData.getString(KEY_USER_ID)
            ?: return Result.failure()
        return try {
            repo.sync(userId)
            Result.success()
        } catch (e: IOException) {
            Result.retry()
        }
    }
}
```

- `Result.success()` → done · `Result.retry()` → run again later, with backoff · `Result.failure()` → stop; dependent work fails too
- `CoroutineWorker` for Kotlin; `Worker` runs blocking code on a background executor



Esmaeeil Moradi
@esmaeeil-moradi700

Q2. Enqueue it — and know its limits

```
val request = OneTimeWorkRequestBuilder<SyncWorker>()
    .setInputData(workDataOf(KEY_USER_ID to userId))
    .setConstraints(
        Constraints.Builder()
            .setRequiredNetworkType(NetworkType.CONNECTED)
            .build()
    )
    .build()

WorkManager.getInstance(context).enqueue(request)
```

INTERVIEW TRAP

"`CoroutineWorker` runs on `Dispatchers.IO`." — It runs on **`Dispatchers.Default`**. Main-safe `suspend` APIs (Room, Retrofit, Ktor) are fine; wrap blocking calls (file streams, legacy SDKs) in `withContext(Dispatchers.IO)`.

INTERVIEW TRAP

A worker isn't unlimited: each run gets **up to 10 minutes**, then it's stopped (the coroutine is cancelled) and re-scheduled. Longer work needs a different approach — Q8.



Esmaeeil Moradi
@esmaeeil-moradi700

Q3. Constraints: run only when it makes sense

Constraint	Builder call
Network	`setRequiredNetworkType(NetworkType.CONNECTED)` — also `UNMETERED`, `NOT_ROAMING`, `METERED`
Charging	`setRequiresCharging(true)`
Battery not low	`setRequiresBatteryNotLow(true)`
Storage not low	`setRequiresStorageNotLow(true)`
Device idle	`setRequiresDeviceIdle(true)` — can delay work for hours
Content changed	`addContentUriTrigger(uri, true)` — e.g. new photos

- If a constraint stops being met **while the worker runs**, WorkManager stops it and runs it again later
- Constraints are AND-ed: every one must be satisfied

INTERVIEW TRAP

Last year's version listed constraints like `DeviceIdle.ON` and `Charging.ON`. **They don't exist.** The real API is the `Constraints.Builder` setters above — invented API names are an instant red flag.



Esmaeeil Moradi
@esmaeeil-moradi700

Q3. Periodic work

```
// Every 6 h, somewhere in the last 1 h of each period
val refresh = PeriodicWorkRequestBuilder<RefreshWorker>(
    6, TimeUnit.HOURS,
    1, TimeUnit.HOURS, // flex window
).setConstraints(unmeteredAndNotLowBattery)
    .build()
```

- Minimum interval: **15 minutes**; minimum flex: **5 minutes**
- Periodic work can't be chained, and it's **never** `SUCCEEDED` — after each run it goes back to `ENQUEUED`

INTERVIEW TRAP

"A 15-minute interval means it runs every 15 minutes." — It's a **minimum**. Doze, your standby bucket and constraints push runs later. Need a precise time → an alarm. Need real-time data → push (Day 16).



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 15

Mid-Level

Chaining · Unique work · Retries & backoff · Observing & testing



Esmaeeil Moradi
@esmaeeil-moradi700

Q4. Chaining work

```
WorkManager.getInstance(context)
    .beginUniqueWork(
        "upload-$albumId",
        ExistingWorkPolicy.KEEP,
        listOf(compressA, compressB), // run in parallel
    )
    .then(upload)   // starts when both succeeded
    .then(notify)
    .enqueue()
```

- A list in `beginWith()` / `beginUniqueWork()` / `then()` runs **in parallel**
- `then()` waits until **every** parent has succeeded
- A parent's `outputData` becomes the child's `inputData`. With several parents, an `InputMerger` combines them: `OverwritingInputMerger` (default) or `ArrayCreatingInputMerger`



Esmaeeil Moradi
@esmaeeil-moradi700

Q4. Where chains surprise people

INTERVIEW TRAP

"Use `parallel()` to run workers side by side." — There is **no** `parallel()` in WorkManager (last year's version used it). Parallelism is a list passed to `beginWith()` / `then()`, or `WorkContinuation.combine()`.

INTERVIEW TRAP

"Put cleanup at the end of the chain so it always runs." — If any step fails or is cancelled, **every dependent step** is marked `FAILED` or `CANCELLED` too. Cleanup that must always happen belongs in the worker's `finally` block, or in separate work.

INTERVIEW TRAP

`Data` is capped at **10 KB** (`Data.MAX_DATA_BYTES`); building a bigger one throws. Pass IDs and URIs — never bitmaps or JSON blobs. The payload lives in Room or a file.



Esmaeeil Moradi
@esmaeeil-moradi700

Q5. Unique work: exactly one of a kind

`ExistingWorkPolicy`	When work with that name already exists
`KEEP`	Ignore the new request
`REPLACE`	Cancel the existing work (even if running), enqueue the new one
`APPEND`	New work runs after it; if the existing work fails, the new work fails too
`APPEND_OR_REPLACE`	Like `APPEND`, but starts a fresh chain if the existing one failed or was cancelled

`ExistingPeriodicWorkPolicy`	Effect
`KEEP`	Keep the existing schedule
`UPDATE`	Update the spec in place; a running worker isn't interrupted
`CANCEL_AND_REENQUEUE`	Cancel and start over (replaces the deprecated `REPLACE`)



Esmaeeil Moradi
@esmaeeil-moradi700

Q5. Enqueue at app start — safely

```
// Safe to call on every app start
WorkManager.getInstance(context).enqueueUniquePeriodicWork(
    "daily-refresh",
    ExistingPeriodicWorkPolicy.UPDATE,
    refresh,
)
```

- **Unique name** → at most one instance. **Tag** → a label on many requests, for observing or cancelling them as a group (`cancelAllWorkByTag`)

INTERVIEW TRAP

`CANCEL_AND_REENQUEUE` (or plain `enqueue()`) in `Application.onCreate()` restarts the schedule — or stacks up duplicates — on every launch, and can cancel a sync that is running. Use `KEEP` or `UPDATE`.



Esmaeeil Moradi
@esmaeeil-moradi700

Q6. Retries and backoff

```
override suspend fun doWork(): Result {
    if (runAttemptCount >= MAX_ATTEMPTS) return Result.failure()
    return try {
        repo.pushPendingChanges()
        Result.success()
    } catch (e: CancellationException) {
        throw e
    } catch (e: IOException) {
        Result.retry()
    } catch (e: HttpException) {
        if (e.code() >= 500) Result.retry()
        else Result.failure()
    }
}
```

```
.setBackoffCriteria(
    BackoffPolicy.EXPONENTIAL, 30, TimeUnit.SECONDS,
)
```



Esmaeeil Moradi
@esmaeeil-moradi700

Q6. What a good retry policy knows

- Default: **exponential, starting at 30 s** (30 s, 1 min, 2 min…), allowed range **10 s to 5 h**; `LINEAR` also exists
- The backoff delay is a **minimum** — constraints and Doze still apply
- `runAttemptCount` lets you give up after N attempts

INTERVIEW TRAP

`runCatching { }` in `doWork()` also catches `CancellationException`. When the system stops your worker (constraint lost, 10-minute limit), it keeps working instead of stopping cooperatively. Rethrow it, or catch specific exceptions.

INTERVIEW TRAP

Retrying everything. A `400` or `401` fails the same way on every attempt — return `failure()` (or fix auth first). Retry only **transient** errors: I/O, 5xx, 429.

RETRIES MEAN DUPLICATES

A retry can re-send a request the server already processed. Send an **idempotency key** with every write.



Esmaeeil Moradi
@esmaeeil-moradi700

Q7. Observing work from the UI

```
// ViewModel
val upload: StateFlow<UploadUi> = workManager
    .getWorkInfosForUniqueWorkFlow("upload-$albumId")
    .map { infos -> infos.firstOrNull().toUploadUi() }
    .stateIn(
        viewModelScope,
        SharingStarted.WhileSubscribed(5_000),
        UploadUi.Idle,
    )

// Worker
setProgress(workDataOf(KEY_PROGRESS to percent))
```

- States: `ENQUEUED` → `RUNNING` → `SUCCEEDED` / `FAILED` / `CANCELLED`; `BLOCKED` = waiting for parents
- `progress` is available only while `RUNNING`; `outputData` only once finished

INTERVIEW TRAP

Treating `WorkInfo` as your data source. Show *status* from it; the *result* comes from the repository (Room) — one source of truth (Day 3). And a periodic worker never reaches `SUCCEEDED`.



Esmaeeil Moradi
@esmaeeil-moradi700

Q7. Testing workers

```
@Test fun retriesOnNetworkError() = runTest {
    val worker = TestListenableWorkerBuilder<SyncWorker>(ctx)
        .setInputData(workDataOf(KEY_USER_ID to "42"))
        .setWorkerFactory(FakeWorkerFactory(OfflineRepo()))
        .build()

    val result = worker.doWork()

    assertEquals(ListenableWorker.Result.retry(), result)
}
```

- `work-testing`: `TestListenableWorkerBuilder` runs `doWork()` directly — set input, `setRunAttemptCount()`, a fake factory
- For requests, chains and constraints: `WorkManagerTestInitHelper` + `TestDriver` (`setAllConstraintsMet(id)`, `setInitialDelayMet(id)`, `setPeriodDelayMet(id)`)

INTERVIEW TRAP

Waiting for real WorkManager with `Thread.sleep()`. Initialize the test WorkManager with a `SynchronousExecutor` and drive time and constraints with `TestDriver`.



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 15

Senior

Expedited & long-running work · Doze & App Standby · Limits by version



Esmaeeil Moradi
@esmaeeil-moradi700

Q8. Expedited work

```
val send = OneTimeWorkRequestBuilder<SendMessageWorker>()
    .setExpedited(
        OutOfQuotaPolicy.RUN_AS_NON_EXPEDITED_WORK_REQUEST
    )
    .build()

class SendMessageWorker(
    ctx: Context,
    params: WorkerParameters,
) : CoroutineWorker(ctx, params) {
    // Below Android 12 it runs as a foreground service
    override suspend fun getForegroundInfo() =
        ForegroundInfo(NOTIFICATION_ID, sendingNotification())

    override suspend fun doWork(): Result = send()
}
```

- Starts **as soon as possible**, even in the background, and is less affected by power-saving deferral
- Android 12+: runs as an **expedited job**, limited by a per-app time **quota**; out of quota → run as normal work, or drop
- Can't be periodic and can't have an initial delay



Esmaeeil Moradi
@esmaeeil-moradi700

Q8. Expedited vs long-running

INTERVIEW TRAP

"Mark everything important as expedited." — The quota is shared. Burn it on analytics and the payment confirmation runs late. Expedited = short tasks the user is waiting for: send a message, confirm a payment.

INTERVIEW TRAP

No `getForegroundInfo()` override → expedited work **crashes on devices below Android 12**, where WorkManager needs it to show the foreground notification.

Longer than 10 minutes? A long-running worker calls `setForeground()` and becomes a foreground service:

```
setForeground(
    ForegroundInfo(
        NOTIFICATION_ID, progressNotification(),
        ServiceInfo.FOREGROUND_SERVICE_TYPE_DATA_SYNC,
    )
)
```

It follows every FGS rule — type declared on WorkManager's `SystemForegroundService` in your manifest (Android 14), background-start limits (Android 12), `dataSync` time limits (Android 15). Details: Day 16.



Esmaeeil Moradi
@esmaeeil-moradi700

Q9. Doze: what stops when the phone sleeps

Doze (Android 6.0) starts when the device is unplugged, stationary and the screen is off; a lighter version (Android 7.0) applies while it's on the move. Work is batched into short **maintenance windows**.

In Doze	Behavior
Network access	Suspended outside maintenance windows
Wake locks	Ignored
Jobs / WorkManager	Deferred to maintenance windows
Regular alarms	Deferred
`setAndAllowWhileIdle()` / `setExactAndAllowWhileIdle()`	Fire, but rate-limited per app
High-priority FCM message	Delivered; app gets a short allowlist window

```
adb shell dumpsys battery unplug
adb shell dumpsys deviceidle force-idle   # enter Doze
adb shell dumpsys deviceidle unforce
adb shell dumpsys battery reset
```



Esmaeeil Moradi
@esmaeeil-moradi700

Q9. App Standby buckets (Android 9+)

The system places each app in a bucket based on how recently and how often it's used:

Bucket	Typical app	Background work
Active	In use right now, or just used	No restrictions
Working set	Used regularly	Jobs and alarms mildly deferred
Frequent	Used often, not daily	Deferred more
Rare	Rarely used	Deferred the most
Restricted (Android 12+)	Unused for long, or very heavy	Roughly once a day

```
adb shell am set-standby-bucket com.example.app rare
adb shell am get-standby-bucket com.example.app
```

- In code: `UsageStatsManager.appStandbyBucket` — log it with your sync metrics

INTERVIEW TRAP

Last year's version listed "Exempted" as a bucket. That's an internal system bucket — apps see the five above. And the answer to deferral is **not** asking every user to disable battery optimization: `REQUEST_IGNORE_BATTERY_OPTIMIZATIONS` is limited by Play policy to specific use cases.



Esmaeeil Moradi
@esmaeeil-moradi700

Q10. Background limits, version by version

Android	What changed for background work
6.0 (API 23)	Doze and App Standby
7.0 (API 24)	Doze on the move; no `CONNECTIVITY_ACTION` for manifest receivers
8.0 (API 26)	Background service limits; implicit-broadcast limits
9 (API 28)	App Standby buckets
12 (API 31)	Restricted bucket; expedited jobs; FGS start limits; exact-alarm permission
14 (API 34)	Mandatory FGS types; user-initiated data transfer jobs
15 (API 35)	`dataSync` FGS time limit; FGS limits after boot
16 (API 36)	Job runtime quotas depend on standby bucket and app state

INTERVIEW TRAP

"`startService()` from a receiver is fine if the work is quick." — Since Android 8.0 an app in the background gets an `IllegalStateException`. Hand the work to WorkManager.

ANSWER WITH THE VERSION

"That's an Android 12 rule for apps targeting 31+" beats "Android forbids it". Most limits follow **targetSdk**; Doze and buckets hit every app.



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 15



Architect

A background-work layer · Battery-friendly sync · Observability



Esmaeeil Moradi
@esmaeeil-moradi700

Q11. Designing the background-work layer

```
// domain — no WorkManager imports
interface SyncScheduler {
    fun requestSync()
}

// data — the only place that knows WorkManager
@HiltWorker
class SyncWorker @AssistedInject constructor(
    @Assisted context: Context,
    @Assisted params: WorkerParameters,
    private val repo: SyncRepository,
) : CoroutineWorker(context, params) {
    override suspend fun doWork(): Result =
        repo.drainPendingChanges().toWorkResult()
}
```

- Workers are **thin**: read input, call a repository, map the outcome to a `Result`
- Work is **idempotent** — it can run twice. Pass IDs; the state lives in Room
- Hilt: the `Application` implements `Configuration.Provider` with `HiltWorkerFactory`



Esmaeeil Moradi
@esmaeeil-moradi700

Q11. The traps that hit at scale

INTERVIEW TRAP

Custom `WorkerFactory` but the default initializer is still in the manifest → WorkManager starts with the default factory and your injected workers can't be created. Remove `WorkManagerInitializer` from the `InitializationProvider`.

INTERVIEW TRAP

Renaming or moving a worker class. Enqueued requests store the **class name**; after the update WorkManager can't instantiate it and the work fails. Keep the old class as a delegate for a release, or cancel and re-enqueue by unique name on upgrade.

INTERVIEW TRAP

`KEEP` silently drops a sync requested **while one is running** — changes made mid-run wait for the next trigger. Let the worker drain the pending queue from Room until it's empty.



Esmaeeil Moradi
@esmaeeil-moradi700

Q12. Battery-friendly sync strategy

Pattern	Why
Push-to-sync: FCM message → one-time work	Syncs only when something changed
Periodic safety net: hours, not minutes	Catches missed pushes; heavy work on unmetered + charging
Coalesce with unique work	One wake-up instead of ten
Sync on app open	The user is here and the device is awake
Server-tuned intervals (Remote Config)	Change cadence without a release

```
val nightly = PeriodicWorkRequestBuilder<BackupWorker>(
    24, TimeUnit.HOURS,
).setInitialDelay(Random.nextLong(0, 60), TimeUnit.MINUTES)
    .setConstraints(unmeteredAndCharging)
    .build()
```

INTERVIEW TRAP

Every client syncing at 00:00 — a self-made DDoS on your backend. Add jitter, let WorkManager spread runs, and honor `Retry-After`. And a 15-minute poll is never "real time": use push.



Esmaeeil Moradi
@esmaeeil-moradi700

Q13. Observability: debugging work you can't see

Question	Tool
Is my work enqueued, blocked, running?	Android Studio **Background Task Inspector**
Why isn't it running?	`adb shell dumpsys jobscheduler`
WorkManager's own view	Diagnostics broadcast below — debug builds, output in Logcat
Run a job now	`adb shell cmd jobscheduler run -f <pkg> <jobId>`
Behavior when rarely used	`am set-standby-bucket`, `deviceidle force-idle` (Q9)
Battery impact in the field	Android vitals: excessive wake-ups, wake locks

```
adb shell am broadcast -p com.example.app \
    -a androidx.work.diagnostics.REQUEST_DIAGNOSTICS
```

INTERVIEW TRAP

"It works on my phone." — Your phone is charging, on Wi-Fi, in the Active bucket. Real users are in Rare, on metered data, in Doze. Test those states on purpose.



Esmaeeil Moradi
@esmaeeil-moradi700

Q13. Production metrics from every worker

```kotlin
abstract class TrackedWorker(
    ctx: Context, params: WorkerParameters,
    private val metrics: WorkMetrics,
) : CoroutineWorker(ctx, params) {

    protected abstract suspend fun work(): Result

    final override suspend fun doWork(): Result {
        val name = this::class.simpleName.orEmpty()
        return try {
            val result = work()
            metrics.finished(name, runAttemptCount, result)
            result
        } catch (e: CancellationException) {
            metrics.stopped(name, runAttemptCount)
            throw e
        }
    }
}
```

- Track success/failure/stop rates, attempts per success, and **enqueue → start latency** — split by standby bucket and Android version

SENIOR ANSWER IN ONE LINE

"WorkManager guarantees the work is *remembered*, not *when* it runs. I design for late, repeated and stopped execution — idempotent workers, bounded retries, and metrics that show how late."



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 15 · ARCHITECT

Day 15 — cheat sheet

- WorkManager = **persistent** work that survives process death and reboot; JobScheduler underneath.
- Force stop cancels everything until the app is opened again — "guaranteed" means *eventually*.
- `CoroutineWorker` runs on `Dispatchers.Default`; each run has up to **10 minutes**.
- Constraints are `Constraints.Builder` setters; periodic work: **15 min** minimum, never `SUCCEEDED`.
- Chains: lists run in parallel; a failure fails every dependent. `Data` ≤ 10 KB.
- Unique work: `KEEP` / `UPDATE` at app start, never re-enqueue blindly.
- Retry only transient errors; rethrow `CancellationException`; send idempotency keys.
- Expedited = urgent and short, limited by quota; needs `getForegroundInfo()` below Android 12.
- Doze and standby buckets decide **when** work runs — test them with `adb`.
- Architect: thin idempotent workers behind an interface, push-to-sync + periodic safety net, metrics per worker.

Day 16 is next → Background Work, Part 2: services, foreground services, alarms & push. Save this post and follow along.



Esmaeeil Moradi
@esmaeeil-moradi700